```python
#basic operation and statistics

import numpy as np

a=2
b=4

print("addition:",a+b)
print("subtraction:",a-b)
print("multiplication:",a*b)
print("division:",a/b)

data=np.array([1,2,3,4,4])
mean=np.mean(data)
median=np.median(data)
population_variance=np.var(data)
sample_variance=np.var(data,ddof=1)

print("mean:",mean)
print("median:",median)
print("population_variance:",population_variance)
print("sample_variance:",sample_variance)
```

```
addition: 6
subtraction: -2
multiplication: 8
division: 0.5
mean: 2.8
median: 3.0
population_variance: 1.3599999999999999
sample_variance: 1.7
```

```python
#csv file & perform basic operation(filter, sort)

import pandas as pd  #for data analysis and manipulationn
import numpy as np  #for numerical computation

data=pd.read_csv("student.csv")
print(data)

df_filter=data[data['percentage']>30]
print("\nfilter percentage > 30:")
print(df_filter)

sorted_by_name=data.sort_values(by='student name')
print("/nSorted by student name:")
print(sorted_by_name)

sorted_by_rno=data.sort_values(by=['roll no'],ascending=False)
print("/nSorted by roll no:")
print(sorted_by_rno)
```

```python
#extra
sorted_by_rno=data.sort_values(by='roll no')
print("/nSorted by roll no:")
print(sorted_by_rno)
```

```
   roll no student name      city  percentage
0        1            a      pune          20
1        2            s    nashik          43
2        3            d   jalgaon          54
3        4            g   pachora          76
4        5            r     dhule          46
5        6            b    mumbai          58
6        7            m   shegaon          25
7        8            v   amalner          14
8        9            c  bhusawal          68
9       10            x    parola          90

filter percentage > 30:
   roll no student name      city  percentage
1        2            s    nashik          43
2        3            d   jalgaon          54
3        4            g   pachora          76
4        5            r     dhule          46
5        6            b    mumbai          58
8        9            c  bhusawal          68
9       10            x    parola          90
/nSorted by student name:
   roll no student name      city  percentage
0        1            a      pune          20
5        6            b    mumbai          58
8        9            c  bhusawal          68
2        3            d   jalgaon          54
3        4            g   pachora          76
6        7            m   shegaon          25
4        5            r     dhule          46
1        2            s    nashik          43
7        8            v   amalner          14
9       10            x    parola          90
/nSorted by roll no:
   roll no student name      city  percentage
9       10            x    parola          90
8        9            c  bhusawal          68
7        8            v   amalner          14
6        7            m   shegaon          25
5        6            b    mumbai          58
4        5            r     dhule          46
3        4            g   pachora          76
2        3            d   jalgaon          54
1        2            s    nashik          43
```

```
0         1            a      pune          20
/nSorted by roll no:
   roll no student name      city  percentage
0         1            a      pune          20
1         2            s    nashik          43
2         3            d   jalgaon          54
3         4            g   pachora          76
4         5            r     dhule          46
5         6            b    mumbai          58
6         7            m   shegaon          25
7         8            v   amalner          14
8         9            c  bhusawal          68
9        10            x    parola          90
```

```
#Normalize and standardize and standardize numerical data using scikit learn

import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler,StandardScaler

data={
    'A':[1,2,3,4,5],
    'B':[11,25,32,44,58],
    'C':[100,200,300,400,500]
}

df=pd.DataFrame(data)
print("Original Data.")
print(df)

min_max_scaler=MinMaxScaler()
normalized_data=min_max_scaler.fit_transform(df)
df_normalized=pd.DataFrame(normalized_data,columns=df.columns)
print("\nNormalized Data(MinMaxScaling):")
print(df_normalized)

Standard_Scaler=StandardScaler()
Standardized_data=Standard_Scaler.fit_transform(df)
df_Standardized=pd.DataFrame(Standardized_data,columns=df.columns)
print("\nStandardized Data:(Z-score Scaling.)")
print(df_Standardized)
```

```
Original Data.
   A   B    C
0  1  11  100
1  2  25  200
2  3  32  300
3  4  44  400
4  5  58  500
```

```
Normalized Data(MinMaxScaling):
      A         B     C
0  0.00  0.000000  0.00
1  0.25  0.297872  0.25
2  0.50  0.446809  0.50
3  0.75  0.702128  0.75
4  1.00  1.000000  1.00

Standardized Data:(Z-score Scaling.)
          A         B         C
0 -1.414214 -1.431917 -1.414214
1 -0.707107 -0.560316 -0.707107
2  0.000000 -0.124515  0.000000
3  0.707107  0.622573  0.707107
4  1.414214  1.494175  1.414214
```

```
#Onehot Encoding and Label Encoding

import pandas as pd
from sklearn.preprocessing import OneHotEncoder, LabelEncoder

data={
    'Category':['A','B','A','C','B','C'],
    'Values':[10,20,10,30,20,30]
}
df=pd.DataFrame(data)
print("Original DataFrame")
print(df)

one_hot_encoder_df=pd.get_dummies(df,columns=['Category'])
print("\nOne Hot Encoded Data")
print(one_hot_encoder_df)

Label_Encoder=LabelEncoder()
df["Category_Label"]=Label_Encoder.fit_transform(df['Category'])
print("\nLabel Encoded Data")
print(df)
```

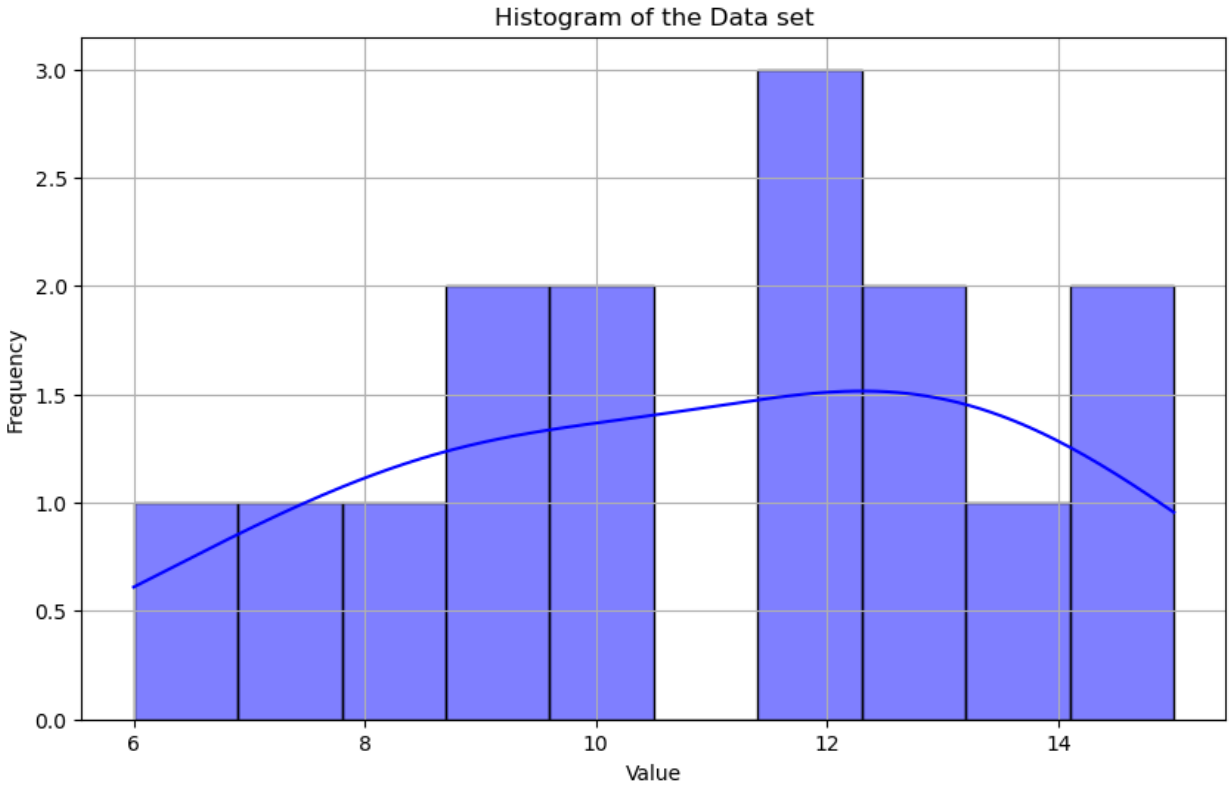
```
Original DataFrame
  Category  Values
0        A      10
1        B      20
2        A      10
3        C      30
4        B      20
5        C      30

One Hot Encoded Data
   Values  Category_A  Category_B  Category_C
```

```
0      10        True       False       False
1      20       False        True       False
2      10        True       False       False
3      30       False       False        True
4      20       False        True       False
5      30       False       False        True
```

```
Label Encoded Data
  Category  Values  Category_Label
0        A      10               0
1        B      20               1
2        A      10               0
3        C      30               2
4        B      20               1
5        C      30               2
```

```
#Histogram for Skewness and Kurtosis

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew,kurtosis

data=[12,10,13,15,7,9,12,14,15,10,9,6,8,12,13]

data_series=pd.Series(data)
data_skewness=skew(data_series)
data_kurtosis=kurtosis(data_series)

print(f"skewness:{data_skewness}")
print(f"kurtosis:{data_kurtosis}")

plt.figure(figsize=(10,6))
sns.histplot(data_series,bins=10,kde=True,color='blue')
plt.title('Histogram of the Data set')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```

```
skewness:-0.19605135394055354
kurtosis:-1.0580357142857144
```




```
#One-Sample T-Test

import numpy as np
import pandas as pd
from scipy import stats

sample_data_one = [12, 15, 14, 10, 8, 15, 14, 12, 10, 9]
population_mean = 12

t_stat_one, p_value_one = stats.ttest_1samp(sample_data_one,
population_mean)


print("One-Sample T-Test:")
print(f"T-statistic: {t_stat_one}, P-value: {p_value_one}")
if p_value_one < 0.05:
    print("Reject the null hypothesis: The sample mean is
significantly different from the population mean.")
else:
    print("Fail to reject the null hypothesis: The sample mean is not
significantly different from the population mean.")
```

```
One-Sample T-Test:
T-statistic: -0.12361284651454894, P-value: 0.9043384285084789
Fail to reject the null hypothesis: The sample mean is not
significantly different from the population mean.
```

```
#Independent Two-Sample T-Test

import numpy as np
import pandas as pd
from scipy import stats

sample_data_a = [12, 15, 14, 10, 8]
sample_data_b = [14, 15, 13, 17, 19]

t_stat_two, p_value_two = stats.ttest_ind(sample_data_a,
sample_data_b)

print("\nIndependent Two-Sample T-Test:")
print(f"T-statistic: {t_stat_two}, P-value: {p_value_two}")
if p_value_two < 0.05:
 print("Reject the null hypothesis: The means of the two samples are
significantly different.")
else:
 print("Fail to reject the null hypothesis: The means of the two
samples are not significantly different.")


Independent Two-Sample T-Test:
T-statistic: -2.270934357735347, P-value: 0.05281335911209561
Fail to reject the null hypothesis: The means of the two samples are
not significantly different.

#one way anova

import numpy as np
import scipy.stats as stats
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm

group_A = [89, 89, 88, 78, 79]
group_B = [93, 92, 94, 89, 88]
group_C = [89, 88, 89, 93, 90]

f_statistic, p_value = stats.f_oneway(group_A, group_B, group_C)
print("\n----- One-Way ANOVA -----")
print(f"F-statistic: {f_statistic}")
print(f"P-value: {p_value}")

alpha = 0.05
if p_value < alpha:
 print("Reject the null hypothesis: At least one of the group means is
significantly different.")
else:
```

```
 print("Fail to reject the null hypothesis: The group means are not
significantly different.")


----- One-Way ANOVA -----
F-statistic: 4.35011990407674
P-value: 0.03795204795237708
Reject the null hypothesis: At least one of the group means is
significantly different.

#two way anova

import numpy as np
import scipy.stats as stats
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm

data = {
 'FactorA': ['A1', 'A1', 'A1', 'A2', 'A2', 'A2', 'A1', 'A1', 'A1',
'A2', 'A2', 'A2'],
 'FactorB': ['B1', 'B2', 'B1', 'B2', 'B1', 'B2', 'B1', 'B2', 'B1',
'B2', 'B1', 'B2'],
 'Value': [23, 21, 25, 30, 28, 31, 22, 23, 21, 35, 34, 32]
}
df = pd.DataFrame(data)
print(df)

model = ols('Value ~ C(FactorA) + C(FactorB) + C(FactorA):C(FactorB)',
data=df).fit()
anova_result = anova_lm(model)
print("Two-Way ANOVA Results:")
print(anova_result)
alpha = 0.05
if anova_result['PR(>F)'][0] < alpha:
    print("FactorA has a significant effect on the dependent
variable.")
else:
 print("FactorA does not have a significant effect on the dependent
variable.")
if anova_result['PR(>F)'][1] < alpha:
 print("FactorB has a significant effect on the dependent variable.")
else:
 print("FactorB does not have a significant effect on the dependent
variable.")
if anova_result['PR(>F)'][2] < alpha:
 print("There is a significant interaction effect between FactorA and
FactorB.")
else:
```

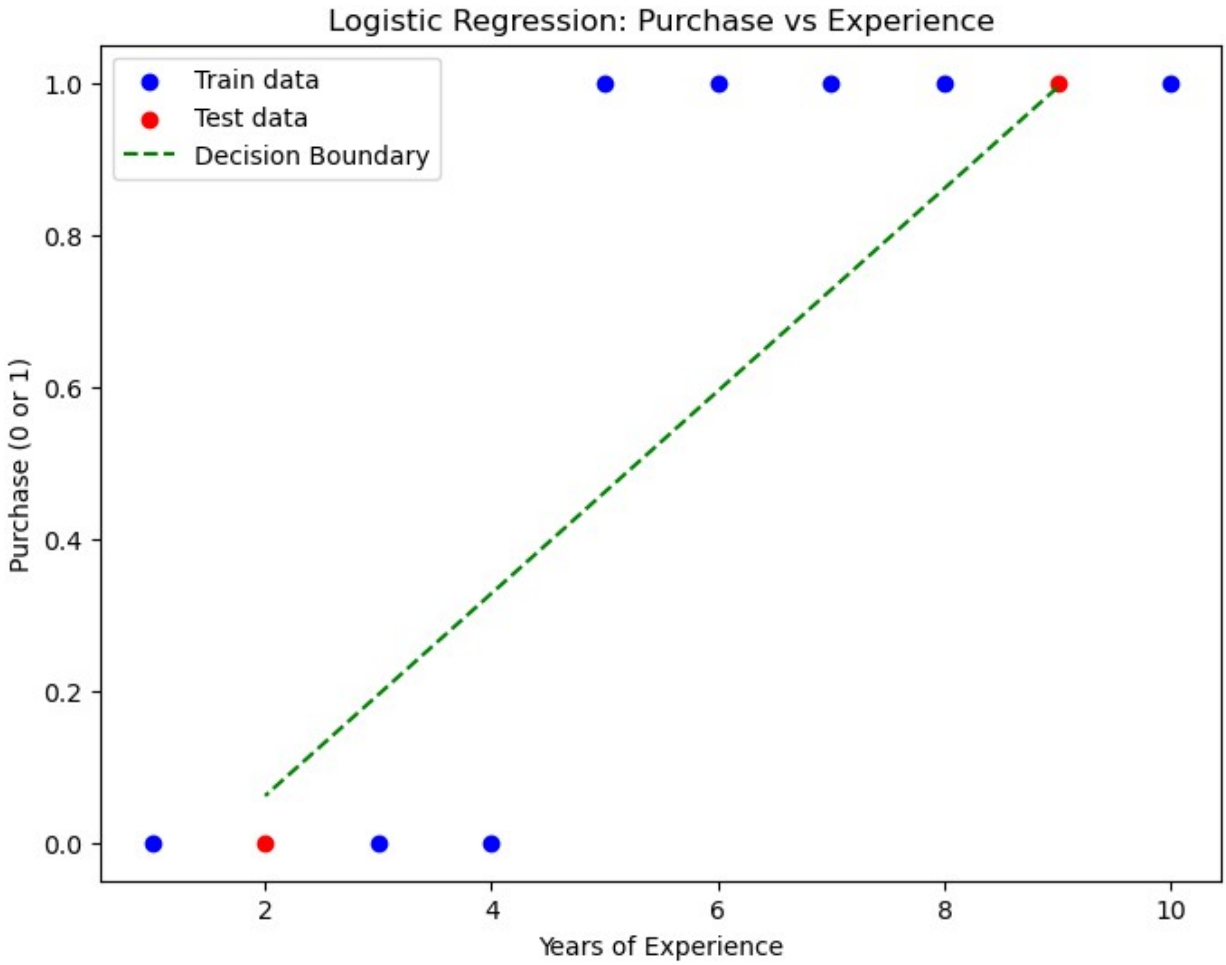
```
 print("There is no significant interaction effect between FactorA and FactorB.")
```

```
   FactorA FactorB  Value
0       A1      B1     23
1       A1      B2     21
2       A1      B1     25
3       A2      B2     30
4       A2      B1     28
5       A2      B2     31
6       A1      B1     22
7       A1      B2     23
8       A1      B1     21
9       A2      B2     35
10      A2      B1     34
11      A2      B2     32
Two-Way ANOVA Results:
                         df      sum_sq     mean_sq          F
PR(>F)
C(FactorA)              1.0  252.083333  252.083333  47.173489
0.000129
C(FactorB)              1.0    0.041667    0.041667   0.007797
0.931807
C(FactorA):C(FactorB)  1.0    2.041667    2.041667   0.382066
0.553685
Residual                8.0   42.750000    5.343750        NaN
NaN
FactorA has a significant effect on the dependent variable.
FactorB does not have a significant effect on the dependent variable.
There is no significant interaction effect between FactorA and FactorB.

C:\Users\ACER\AppData\Local\Temp\ipykernel_3756\1962895551.py:21:
FutureWarning: Series.__getitem__ treating keys as positions is
deprecated. In a future version, integer keys will always be treated
as labels (consistent with DataFrame behavior). To access a value by
position, use `ser.iloc[pos]`
  if anova_result['PR(>F)'][0] < alpha:
C:\Users\ACER\AppData\Local\Temp\ipykernel_3756\1962895551.py:25:
FutureWarning: Series.__getitem__ treating keys as positions is
deprecated. In a future version, integer keys will always be treated
as labels (consistent with DataFrame behavior). To access a value by
position, use `ser.iloc[pos]`
  if anova_result['PR(>F)'][1] < alpha:
C:\Users\ACER\AppData\Local\Temp\ipykernel_3756\1962895551.py:29:
FutureWarning: Series.__getitem__ treating keys as positions is
deprecated. In a future version, integer keys will always be treated
as labels (consistent with DataFrame behavior). To access a value by
position, use `ser.iloc[pos]`
  if anova_result['PR(>F)'][2] < alpha:
```

```python
#logistic regression

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns

data = {
 'X': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
 'y': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df[['X']]
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print("\n----- Logistic Regression -----")
print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")

plt.figure(figsize=(8, 6))
plt.scatter(X_train, y_train, color='blue', label='Train data')
plt.scatter(X_test, y_test, color='red', label='Test data')
plt.plot(X_test, model.predict_proba(X_test)[:,1], color='green',
label='Decision Boundary', linestyle='--')
plt.xlabel('Years of Experience')
plt.ylabel('Purchase (0 or 1)')
plt.title('Logistic Regression: Purchase vs Experience')
plt.legend()
plt.show()
```

```
----- Logistic Regression -----
Accuracy: 1.0
Confusion Matrix:
```

```
[[1 0]
 [0 1]]
```


Logistic Regression: Purchase vs Experience
Train data
Test data
Decision Boundary
Purchase (0 or 1)
Years of Experience
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
10